

EX.NO.: 02

DATE: 24.12.2024

DCGAN

AIM

To train a Deep Convolutional Generative Adversarial Network (DCGAN) to generate realistic synthetic images of celebrities.

DATASET DESCRIPTION

Dataset Name: CelebA (Large-scale CelebFaces Attributes Dataset)

Description:

- The CelebA dataset consists of over 200,000 celebrity images.
- Each image is annotated with 40 attribute labels such as age, gender, and facial features.
- The images cover large pose variations and background clutter.
- The dataset is widely used for various computer vision tasks, including face detection, attribute prediction, and image generation.

TECHNIQUES USED

1. **Generative Adversarial Networks (GANs):**
 - a. GANs consist of two neural networks, a generator and a discriminator, that are trained simultaneously through adversarial learning.
 - b. The generator creates synthetic data samples, while the discriminator evaluates their authenticity compared to real data.
2. **Deep Convolutional Generative Adversarial Network (DCGAN):**
 - a. A DCGAN is a type of GAN that employs convolutional layers in both the generator and discriminator.
 - b. DCGANs are known for their ability to generate high-quality images.
3. **TensorFlow and Keras:**
 - a. TensorFlow is an open-source machine learning framework used for implementing and training the DCGAN model.
 - b. Keras is a high-level API within TensorFlow that simplifies the process of building and training deep learning models.

ARCHITECTURE

Generator Architecture:

- Input: Latent vector (random noise).
- Layers: Dense layer -> Reshape -> BatchNorm -> LeakyReLU -> Conv2DTranspose (Deconvolution) layers.
- Output: Generated image of size 64x64x3.

Discriminator Architecture:

- Input: Image of size 64x64x3.
- Layers: Conv2D -> LeakyReLU -> Dropout -> Flatten -> Dense layer.
- Output: Single scalar indicating real or fake.

CODE AND OUTPUT

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import matplotlib.pyplot as plt
import os
import time

# Download and unzip the dataset
!wget https://s3-us-west-1.amazonaws.com/udacity-dlnfd/datasets/celeba.zip
```

```
!unzip celeba.zip -d celeba_gan
```

```
# Define image parameters
```

```
IMG_HEIGHT = 64
```

```
IMG_WIDTH = 64
```

```
BATCH_SIZE = 32
```

```
latent_dim = 100
```

```
# Load and preprocess the dataset
```

```
dataset = keras.utils.image_dataset_from_directory(
```

```
    "celeba_gan/img_align_celeba", label_mode=None, image_size=(IMG_HEIGHT, IMG_WIDTH),  
    batch_size=BATCH_SIZE
```

```
)
```

```
dataset = dataset.map(lambda x: (x - 127.5) / 127.5) # Normalize to [-1, 1]
```

```
extracting: celeba_gan/img_align_celeba/200693.jpg  
extracting: celeba_gan/img_align_celeba/200694.jpg  
extracting: celeba_gan/img_align_celeba/200695.jpg  
extracting: celeba_gan/img_align_celeba/200696.jpg  
extracting: celeba_gan/img_align_celeba/200697.jpg  
extracting: celeba_gan/img_align_celeba/200698.jpg  
extracting: celeba_gan/img_align_celeba/200699.jpg  
extracting: celeba_gan/img_align_celeba/200700.jpg  
extracting: celeba_gan/img_align_celeba/200701.jpg  
extracting: celeba_gan/img_align_celeba/200702.jpg  
extracting: celeba_gan/img_align_celeba/200703.jpg  
extracting: celeba_gan/img_align_celeba/200704.jpg  
extracting: celeba_gan/img_align_celeba/200705.jpg  
extracting: celeba_gan/img_align_celeba/200706.jpg  
extracting: celeba_gan/img_align_celeba/200707.jpg
```

```
# Define the generator model
```

```
def make_generator_model():
```

```
    model = tf.keras.Sequential()
```

```
    model.add(layers.Dense(8*8*256, use_bias=False, input_shape=(latent_dim,)))
```

```
    model.add(layers.BatchNormalization())
```

```
    model.add(layers.LeakyReLU())
```

```
    model.add(layers.Reshape((8, 8, 256)))
```

```
    assert model.output_shape == (None, 8, 8, 256)
```

```
    model.add(layers.Conv2DTranspose(128, (5, 5), strides=(2, 2), padding='same',  
use_bias=False))
```

```
    assert model.output_shape == (None, 16, 16, 128)
```

```
    model.add(layers.BatchNormalization())
```

```
    model.add(layers.LeakyReLU())
```

```
    model.add(layers.Conv2DTranspose(64, (5, 5), strides=(2, 2), padding='same',  
use_bias=False))
```

```
    assert model.output_shape == (None, 32, 32, 64)
```

```
    model.add(layers.BatchNormalization())
```

```
    model.add(layers.LeakyReLU())
```

```
    model.add(layers.Conv2DTranspose(3, (5, 5), strides=(2, 2), padding='same',  
use_bias=False, activation='tanh'))
```

```
    assert model.output_shape == (None, 64, 64, 3)
```

```

    return model

# Define the discriminator model
def make_discriminator_model():
    model = tf.keras.Sequential()
    model.add(layers.Conv2D(64, (5, 5), strides=(2, 2), padding='same',
                             input_shape=[IMG_HEIGHT, IMG_WIDTH, 3]))

    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))

    model.add(layers.Conv2D(128, (5, 5), strides=(2, 2), padding='same'))
    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))

    model.add(layers.Flatten())
    model.add(layers.Dense(1))

    return model

# Create the generator and discriminator
generator = make_generator_model()
discriminator = make_discriminator_model()

# Define the loss function and optimizers
cross_entropy = tf.keras.losses.BinaryCrossentropy(from_logits=True)

def discriminator_loss(real_output, fake_output):
    real_loss = cross_entropy(tf.ones_like(real_output), real_output)
    fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)
    total_loss = real_loss + fake_loss
    return total_loss

def generator_loss(fake_output):
    return cross_entropy(tf.ones_like(fake_output), fake_output)

generator_optimizer = tf.keras.optimizers.Adam(1e-4)
discriminator_optimizer = tf.keras.optimizers.Adam(1e-4)

# Training loop
@tf.function
def train_step(images):
    noise = tf.random.normal([BATCH_SIZE, latent_dim])

    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
        generated_images = generator(noise, training=True)

        real_output = discriminator(images, training=True)
        fake_output = discriminator(generated_images, training=True)

        gen_loss = generator_loss(fake_output)
        disc_loss = discriminator_loss(real_output, fake_output)

```

```

    gradients_of_generator = gen_tape.gradient(gen_loss, generator.trainable_variables)
    gradients_of_discriminator = disc_tape.gradient(disc_loss,
discriminator.trainable_variables)

    generator_optimizer.apply_gradients(zip(gradients_of_generator,
generator.trainable_variables))
    discriminator_optimizer.apply_gradients(zip(gradients_of_discriminator,
discriminator.trainable_variables))

def train(dataset, epochs):
    for epoch in range(epochs):
        start = time.time()

        for image_batch in dataset:
            train_step(image_batch)

        display_generated_images(generator, latent_dim)

        print(f'Time for epoch {epoch + 1} is {time.time()-start} sec')
# Function to generate and display images
def display_generated_images(generator, latent_dim, num_images=10, figsize=(10, 10)):
    # Generate random latent vectors
    random_latent_vectors = tf.random.normal(shape=(num_images, latent_dim))

    # Generate images
    generated_images = generator(random_latent_vectors, training=False)

    # Rescale pixel values to [0, 1]
    generated_images = (generated_images + 1) / 2.0

    # Plot the images
    plt.figure(figsize=figsize)
    for i in range(num_images):
        plt.subplot(1, num_images, i + 1)
        plt.imshow(generated_images[i])
        plt.axis("off")
    plt.show()

# Train the model
EPOCHS = 10
train(dataset, EPOCHS)

```



Time for epoch 1 is 185.26812934875488 sec



Time for epoch 2 is 183.62671971321106 sec



Time for epoch 3 is 184.82758498191833 sec



Time for epoch 4 is 184.91444325447083 sec



Time for epoch 5 is 184.9370129108429 sec



Time for epoch 6 is 185.18138313293457 sec



Time for epoch 7 is 184.67802596092224 sec



Time for epoch 8 is 184.8410680294037 sec



Time for epoch 9 is 184.77710914611816 sec



Time for epoch 10 is 184.88848042488098 sec

INFERENCE

The trained DCGAN model learns to capture the distribution of the training images, allowing it to generate high-quality, realistic images.